

Playbook (de ponta a ponta)

Primeiro microserviço do sistema de gestão de eventos + governança GitHub (PR/CI/Proteção da main). Este guia já nasce preparado para escalar para os próximos microserviços.

Nota: esta versão para a lousa removeu apenas marcadores internos de citação automática; todo o conteúdo técnico e organizacional foi preservado.

1) Visão geral do que vamos construir

- **Microserviço:** `users-service` (NestJS + TypeORM + PostgreSQL), borda HTTP REST, com Swagger em `/api/docs`.
- **Endpoints mínimos:** `POST /users`, `GET /users`, `GET /users/:id`, `PUT /users/:id`, `DELETE /users/:id`, `GET /health`.
- **Banco dedicado por serviço** (database-per-service) para desacoplamento.

2) Modelo de repositório e governança (vale para este e para os próximos serviços)

1. **Criar repositório** (ou módulo dentro do monorepo) `users-service`. Use README mínimo e `.gitignore` Node.
2. **Ativar CI simples** com 3 jobs: `lint`, `build`, `test` (GitHub Actions) — é o núcleo dos checks exigidos.
3. **Proteger a main**: exigir PR, exigir os 3 checks verdes, “Require branches to be up to date”, sem force push/bypass.
4. **CODEOWNERS** minimalista por área/módulo e **professor só com Read** (não como code owner).
5. **PR template** em `.github/pull_request_template.md` para padronizar contexto, mudanças, impacto, testes.
6. **Checklist de auditoria** no README (proteções da main e evidências).

Dica (monorepo): use um workflow que **detecta** projetos Node no root/pacotes para o CI não quebrar quando não houver `package.json` naquela pasta, e rode em workspaces quando houver.

3) Passo a passo técnico de implementação

3.1. Bootstrap + Docker

- **Pré-requisitos:** Node 18+, Docker/Compose, Nest CLI.
- **Checklist de aceite (setup):** Docker/Compose ok; Nest + TypeORM + Postgres; `.env` local e `.env.example` versionado.
- **Observação:** manter **Swagger** habilitado e `ValidationPipe` global desde o início, com prefixo `api/`.

3.2. Domínio e persistência

- **Entidade** `User`: UUID como PK, colunas auditáveis, versão otimista, índice único por email, `passwordHash` com `select: false`.
- **Migrations (obrigatório)**: nunca use `synchronize: true` em produção; adote "migrate-then-deploy".
- **Checklist de aceite (dados)**: entidade, decorators, auditoria, `@VersionColumn` ok.

3.3. Contratos, validação e erros

- **DTOs (Create/Update/Response)** + validação com `class-validator`; tratar conflitos (unique email) com `409`.
- **Checklist de aceite (API)**: CRUD completo, validação na borda, erros mapeados, Swagger publicado.
- **Segurança**: hash de senha com `bcrypt`; **não** retornar o hash.

3.4. Healthcheck, testes e qualidade

- `GET /health` verificando DB (obrigatório para CI/CD e monitoramento).
- **Testes**: priorize e2e básicos já no MVP; evolua com cobertura.
- **Performance**: paginação, projeções e evitar N+1; índices em email/nome (incluir em PRs de refinamento).

4) CI/CD mínimo viável (Actions)

Crie `/.github/workflows/ci.yml` com três jobs independentes. Exemplo de estratégia: Node 20, `npm ci`, `--if-present` para tolerar monorepo/variações. Por que assim? **Simplicidade que escala** — o mesmo desenho atende múltiplos serviços.

5) Proteção da `main` (regra e seleção de checks)

1. Após o **primeiro PR** rodar e registrar os contexts `lint`, `build`, `test`, ative a regra: **PR obrigatório, status checks exigidos, Require branches to be up to date, sem bypass/force push**.
2. Se a UI não listar os checks, force pela **GitHub CLI** configurando `required_status_checks` (contexts `lint`, `build`, `test`).
3. **Por que isso importa**: qualidade, estabilidade e auditoria de merges.

6) CODEOWNERS e colaboradores

- **Arquivo**: `/.github/CODEOWNERS` (preferido). **Não** inclua o professor como owner se for apenas leitor.
- **Modelos**: por área e/ou por módulo (ex.: `/src/users/** @org/devs @aluno1`).
- **Papéis e acesso**: professor = **Read**, revisores/devs = **Write**; princípio do menor privilégio.
- **Validação**: habilite "Require review from Code Owners" se fizer sentido.

7) Fluxo de trabalho (branches, commits, PRs)

1. **Branch por entrega** (`feat/users-create`, `fix/users-unique-email`). **Commits pequenos** e descritivos.
 2. **Abrir PR** solicitando revisão; aguardar **checks verdes**; **merge com squash** para manter histórico limpo.
 3. **Evidências**: link do PR com "All checks have passed", print da regra de proteção e do workflow. (Use no relatório/aula.)
-

8) Entregáveis (para aula e para o repositório)

- **README** com snippet "Proteção da `main`" e **checklist de evidências**.
 - **Relatório técnico (1-2 págs)** + **apresentação (5-7 slides)** com contexto, implementação, problemas, resultados e links.
-

9) Definição de Pronto (DoD) — conferir antes do merge

- CRUD de `users` funcional, **Swagger** publicado, **validação & erros** ok.
 - Segurança: senha com **bcrypt** e `passwordHash` **nunca** exposto.
 - Migrations criadas e aplicadas; healthcheck respondendo; e2e básicos.
 - CI com `lint`, `build`, `test`; regra de proteção exigindo PR + checks + up-to-date.
 - README com checklist e prints/links das evidências.
-

10) Escalonando para os próximos microserviços

- **Mesma governança** (CI 3 jobs + proteção da main + CODEOWNERS minimalista).
 - **Database-per-service** como padrão de persistência.
 - **Roadmap de evolução**: paginação por cursor/índices; **Outbox/Sagas** para integrações; **auth/roles**; **observabilidade** (métricas e tracing).
-

Roteiro prático resumido (cola rápida)

1) Repo pronto + CI `lint/build/test`. 2) Primeira branch `feat/users-skeleton` → controllers, services, entity, DTOs, Swagger, healthcheck. 3) Migrations + seed inicial. **Sem** `synchronize:true`. 4) Abrir PR → checks verdes → configurar proteção da `main` exigindo **PR + 3 checks + up-to-date** (UI/CLI). 5) CODEOWNERS + colaboradores (Professor=Read, Devs=Write). 6) PR template e README com checklist/evidências. 7) Merge (squash) e tag de release se necessário.

Próximo passo sugerido

Se quiser, converto este conteúdo em um **guia PDF/MD pronto para sala**, já com modelos de `CODEOWNERS`, `pull_request_template.md` e `ci.yml` inicial, além de um checklist de evidências para os grupos anexarem às entregas.